MONASH University

ETC4500/ETC5450 Advanced R programming

Week 8: Object-oriented Programming

Programming paradigms

Functional programming (W6)

- Functions are created and used like any other object.
- Output should only depend on the function's inputs.

Programming paradigms

Functional programming (W6)

- Functions are created and used like any other object.
- Output should only depend on the function's inputs.

Literate programming

- Natural language is interspersed with code.
- Aimed at prioritising documentation/comments.
- Now used to create reproducible reports/documents.

Programming paradigms

Reactive programming

- Objects are expressed using code based on inputs.
- When inputs change, the object's value updates.

Programming paradigms

Reactive programming

- Objects are expressed using code based on inputs.
- When inputs change, the object's value updates.

Object-oriented programming (W8-W9)

- Functions are associated with object types.
- Methods of the same 'function' produce object-specific output.

Object oriented programming

- **Encapsulation**: Bundles data and methods in a class. Protects object integrity by restricting access to internal states.
- **Abstraction**: Simplifies complexity by exposing only essential features of an object. Makes systems easier to understand and use.
- **Polymorphism**: Allows the same function to operate differently on different object types. Enables flexibility
- **Inheritance**: Enables new class to inherit properties and behaviors from an existing class. Useful for structuring data infrastructure.

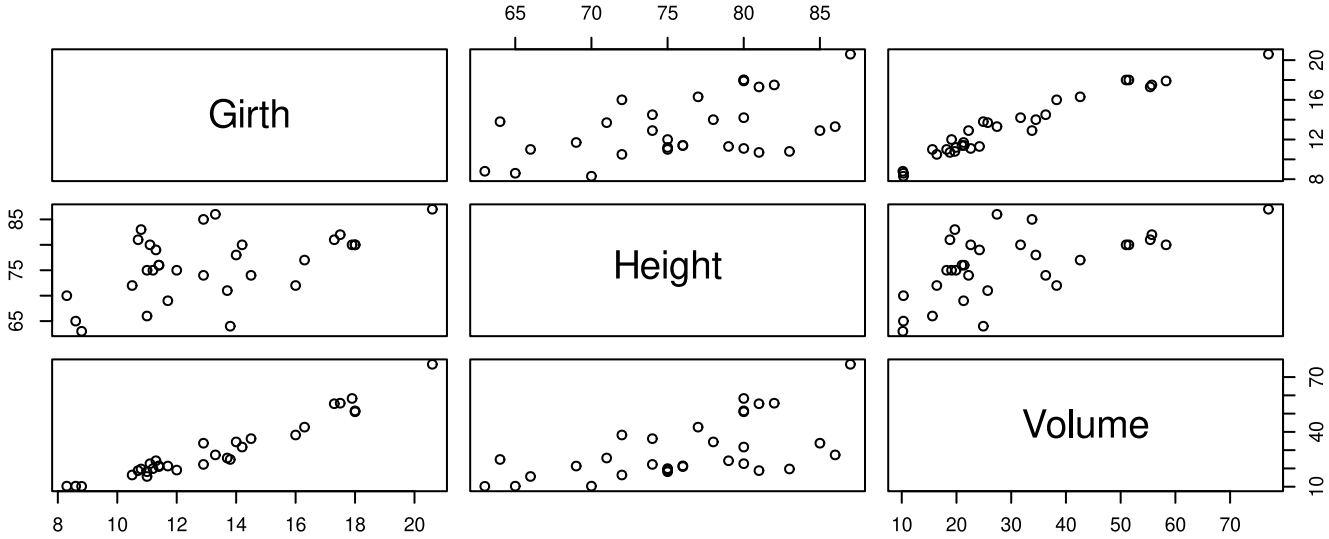
Generic functions and methods

A simple example: `plot`

Generic functions and methods

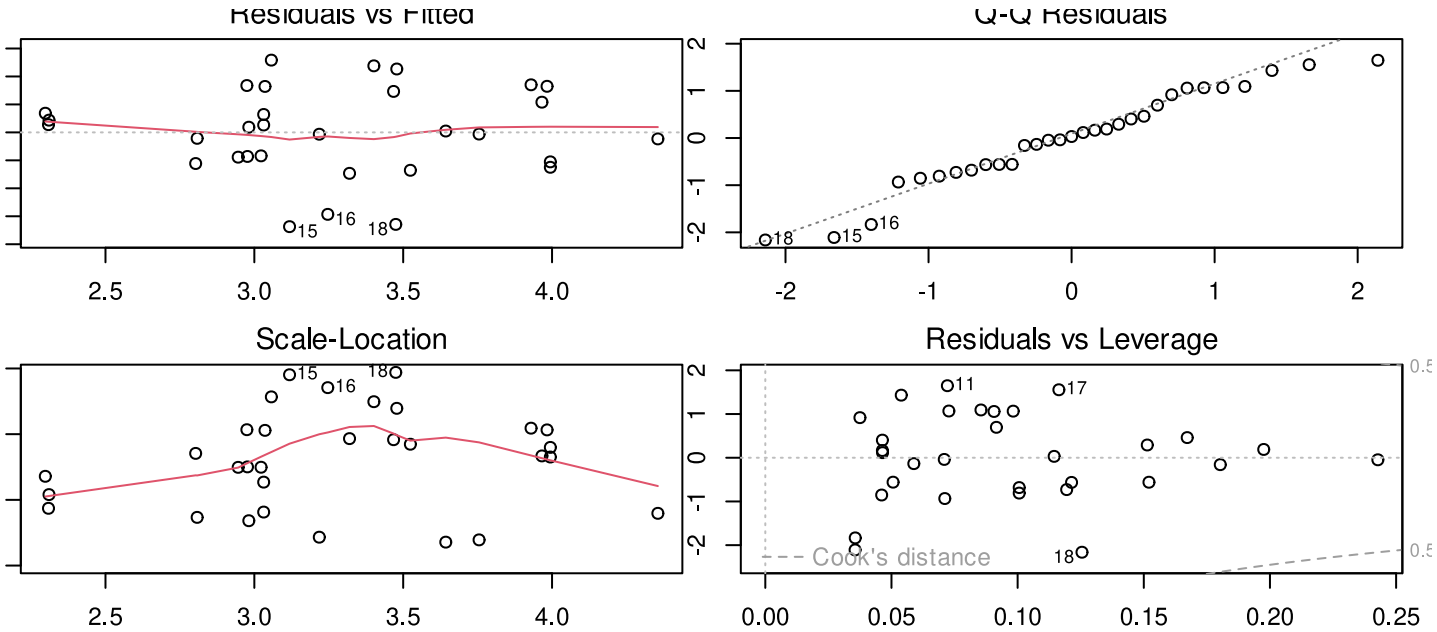
A simple example: `plot`

```
plot(trees)
```



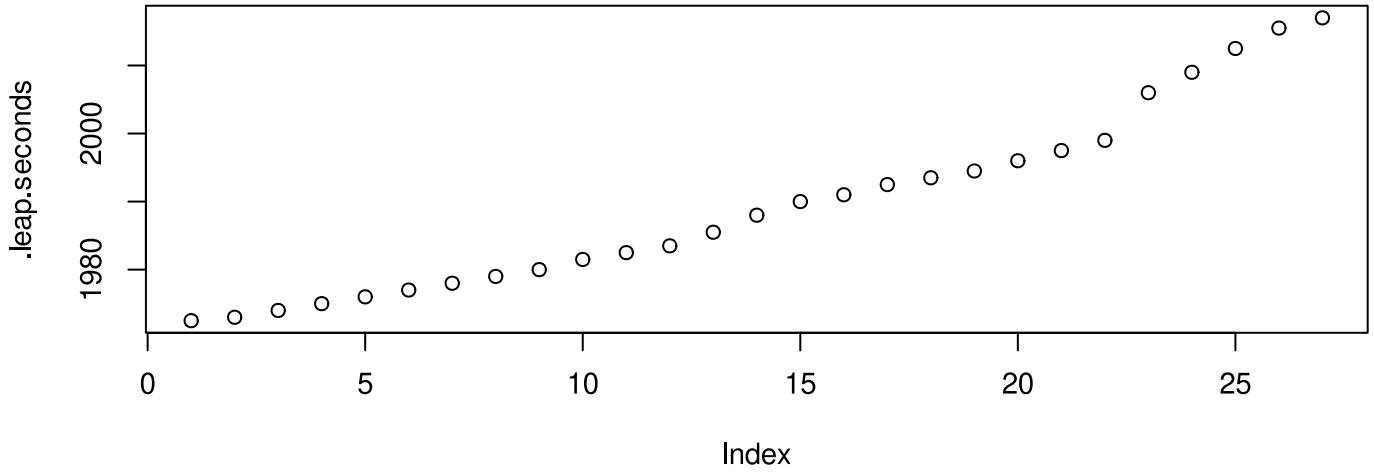
Generic functions and methods

```
m <- lm(log(Volume) ~ log(Girth) + log(Height), data = trees)
par(mfrow = c(2, 2), mar = c(3, 1, 1, 1))
plot(m)
```



Generic functions and methods

```
plot(.leap.seconds)
```



Generic functions and methods

```r
plot(nhtemp)
```

Generic functions and methods

How does `plot()` work?

- Giant `switch` statement...
- Lots of `if` statements...
- How does the behaviour update when you load packages?
- ???

Generic functions and methods

How does `plot()` work?

- Giant `switch` statement...
- Lots of `if` statements...
- How does the behaviour update when you load packages?
- ???

S3 generic functions and methods!

Object systems

R has **a lot** of object systems

- S3
- S3 `vctrs`
- S4
- R6
- R.oo
- proto, ggproto,
- R7, S7

Object oriented programming

S3

- The OO system used by most of CRAN.
- Very simple (and ‘limited’) compared to other systems.
- Informal: no enforcement of types, structure, etc.

Object oriented programming

S3

- The OO system used by most of CRAN.
- Very simple (and ‘limited’) compared to other systems.
- Informal: no enforcement of types, structure, etc.

vctrs

- Builds upon S3 to make creating vectors easier.
- Good practices inherited by default.
- Ensures correct coercion and recycling behaviour across tidyverse operations

Object oriented programming

S4

- Formal class definitions with validation.
- Supports multiple inheritance and method dispatch.
- Used extensively in Bioconductor

Object oriented programming

S4

- Formal class definitions with validation.
- Supports multiple inheritance and method dispatch.
- Used extensively in Bioconductor

S7

- Planned to be the successor of S3 and S4.
- More general than S3, but still easy to use.
- Combines S4-style formal properties with S3-style simplicity and compatibility

Object oriented programming

R6

- Provides reference semantics for mutable objects.
- Simple and efficient compared to reference classes.
- Commonly used for stateful objects such as database connections and loggers

Object oriented programming

R6

- Provides reference semantics for mutable objects.
- Simple and efficient compared to reference classes.
- Commonly used for stateful objects such as database connections and loggers

ggproto

- Used in ggplot2 for extensibility.
- Supports inheritance and method overloading.

S3

Topic for today

- easy to start writing
- no safeguards
- especially good for simple, small-medium projects
- can be used for large projects with a lot of attention to documentation and communication
- limited use of inheritance
- basis of tidyverse and most of CRAN

S3

Back to the `plot` function...

- `plot()` doesn't *do* anything
- All the work is done by *methods* for different *classes*
- Methods are just ordinary functions
- When you call `plot`, R calls the appropriate `plot` method

S3: Generic functions

- Generic functions don't *do* anything
- All the work is done by *methods* for different types of object
- Methods are just ordinary functions
  - with declarations in a package `NAMESPACE`
  - or R can guess based on function name

When you call the generic function R calls the appropriate method

S3: Methods

- *methods* that actually do the work 'belong to' *generic functions*
- This is unusual: most other OOP systems (Java, C++, Python) have methods belonging to data objects
- Important in R because functions are first-class objects (Week 6)
- Useful for functional programming with objects

S3: Classes

- S3 classes are attributes that specify which method to use
- The `class()` function can access (and modify) an object's class
- Classed S3 objects are typically produced with `structure()`

```r
x <- structure(83, class = "grade")
class(x)
```

```
[1] "grade"
```

```r
typeof(x)
```

```
[1] "double"
```

```r
x
```

```
[1] 83
attr(,"class")
[1] "grade"
```

S3: Classed objects

You can class any object, including:

- vectors plus attributes (`ts`, `POSIXct`, `matrix`)
- lists plus attributes (`lm`, `data.frame`)
- environments plus attributes

S3: Your turn!

Use `unclass()` and `str()` to explore classed objects, e.g.:

```
unclass(.leap.seconds)
unclass(nhtemp)
unclass(trees)
m <- lm(log(Volume) ~ log(Girth) + log(Height), data = trees)
str(m)
```

S3: Your turn!

Investigate these functions.

```
print
methods("print")
stats:::print.acf
plot
methods("plot")
plot.ts
stats:::plot.lm
```

How do generic functions relate to methods?

Also, try `methods("plot")` after loading another package.

Writing S3 methods

An S3 method is an ordinary function with some constraints:

- The function's name is of the form `<generic>.<class>`,
- The function's arguments match the generic's arguments,
- The function is registered as an S3 method (for packages).

This looks like:

```
#' Documentation for the method
#' @method <generic> <class>
<generic>.<class> <- function(<generic args>, <method args>, ...) {
  # The code for the method
}
```

S3: Creating a method

```r
print.grade <- function(x, ...) {
  letter <- cut(x,
    breaks = c(-Inf, 50, 60, 70, 80, Inf),
    labels = c("N", "P", "C", "D", "HD"),
    right = FALSE
  )
  output <- paste(x, "[", as.character(letter), "]", sep = "")
  cat(output)
  invisible(x)
}
x
```

```
83[HD]
```

S3: Creating a method

- Create a `plot()` method for `grade` objects that plots a barplot of the grade distribution.
- Create a `summary()` method for `grade` objects that returns a table showing the number and percentage of each letter grade.

Defining methods

- A method should have **the same** arguments as the generic (plus maybe more)
- The name of the method is `paste(generic,class,sep=".")`
- Less ambiguous: use a package and declare the functions
  - `S3method(generic, class)` in `NAMESPACE`
  - `@method generic class` with `devtools`
- `sloop::ftype` tells you about the type of a function
- `sloop::s3_get_method` or `getAnywhere` finds methods even if they're hidden

Adding methods to your class

No *rules* on which methods, but informal standards

- Start with `print`, `[`
- `summary` for more information
- `plot` or `image` if possible (`ggplot` methods take more work)
- `coef`, `vcov`, maybe `logLik` and `AIC` for models
- `resid` for models with residuals

Creating an S3 generic

A typical generic function includes **only** a call to `UseMethod`

```
print
```

```
function (x, ...)
UseMethod("print")
<bytecode: 0x5e4c196b5e68>
<environment: namespace:base>
```

Creating an S3 generic

S3 generics are work like any ordinary function, but they include `UseMethod()` which calls the appropriate method.

Your turn!

Create an S3 generic called “reverse”.

This function will reverse objects. For example,

- `reverse("stressed")` becomes `"desserts"`,
- `reverse(7919)` becomes `9197`,
- `reverse(1.9599)` becomes `9959.1`.

Writing S3 methods

Your turn!

Write methods for reversing `character`, `integer`, and `double` objects.

- `reverse("stressed")` becomes `"desserts"`,
- `reverse(7919L)` becomes `9197L`,
- `reverse(1.9599)` becomes `9959.1`.

Hint: `stringi::stri_reverse()` will reverse a string.

The `integer` and `double` methods should return an integer and double respectively.

S3: `.default` methods

Default methods are called when there is no specific method for the object (no class, or no matching class).

Some examples include:

- `mean.default`
- `quantile.default`
- `summary.default`
- `head.default`

Writing S3 defaults

What if we tried to reverse the current date; `reverse(Sys.Date())`?

Writing S3 defaults

What if we tried to reverse the current date; `reverse(Sys.Date())`?

Your turn!

Question: what should the default behaviour be?

Writing S3 defaults

What if we tried to reverse the current date; `reverse(Sys.Date())`?

Your turn!

Question: what should the default behaviour be?

- Raise an error?
- Return a reversed string?
- Something else entirely?

S3: Defining classes

The S3 class system is simple!

- R doesn't care what `class` you attach to an object
- **You** have to care
- `class(x) <- "lm"` makes R call `lm` methods on `x`
- **You** are responsible for these methods being appropriate
- Documentation is important
- No real enforcement of encapsulation

S3: Defining classes safely

- Document what is in a valid object of your class
- Have a limited set of places where one is created
- Consider having a pure constructor function (ARP 13.3.1)
- Consider having a validator function (ARP 13.3.1)
- Have a user-friendly function to make valid objects

This is especially important if *someone else* might want to make an object from your class

S3: Constructors functions

These functions return classed S3 objects. They should handle input validation and be user-friendly.

Constructor functions typically come in two forms:

- complex: `tibble`, `lm`, `acf`, `svydesign`
- pure: `new_factor`, `new_difftime`

Pure constructor functions simply validate inputs and produce the classed object, while complex constructor functions involve calculations.

Creating your own S3 objects

The `structure()` function is usually used within packages.

- `lm()` returns a list with class `"lm"`, and
- `tibble()` returns a list classed `"tbl_df"`, `"tbl"`, and `"data.frame"`.

Creating your own S3 objects

The `structure()` function is usually used within packages.

- `lm()` returns a list with class `"lm"`, and
- `tibble()` returns a list classed `"tbl_df"`, `"tbl"`, and `"data.frame"`.

Your turn!

Create `fraction()`, which returns `fraction` objects.

The underlying data type is a list containing two vectors for the two arguments: `numerator` and `denominator`.

This function should check that the inputs are suitable.

Creating your own S3 objects

```r
fraction <- function(numerator, denominator) {
  stopifnot(is.numeric(numerator) && is.numeric(denominator))
  stopifnot(length(numerator) == length(denominator))
  if (any(denominator == 0)) {
    stop("Denominator cannot be zero.")
  }

  structure(
    list(numerator = numerator, denominator = denominator),
    class = "fraction"
  )
}
```

Creating your own S3 objects

The `fraction` class doesn't yet have any methods, so it inherits methods from its list type.

```
e <- fraction(numerator = 2721, denominator = 1001)
print(e)
```

```
$numerator
[1] 2721

$denominator
[1] 1001

attr(,"class")
[1] "fraction"
```

Creating your own S3 objects

Usually we would create a method for printing S3 objects.

```
print.fraction <- function(x, ...) {
  cat(paste0(x$numerator, "/", x$denominator), sep = " ")
}
print(e)
```

```
2721/1001
```

Creating your own S3 objects

Your turn!

Create a `reverse()` method for the `fraction` object class, which inverts the numerator and denominator.

Finished early?

Write a method for converting a `fraction` into a number.

S3: Method dispatch

Method dispatch describes the process of calling the appropriate method for the object's class.

Mostly matches `class()`, but not always for some primitive object types. `sloop::s3_class()` shows the extra s3 dispatch classes.

```
> sloop::s3_class(1)
[1] "double"  "numeric"
> sloop::s3_class(matrix(1,1,1))
[1] "matrix"  "double"  "numeric"
> class(1)
[1] "numeric"
> class(matrix(1,1,1))
[1] "matrix" "array"
```

S3: Inheritance

The `class` attribute of an object can have multiple elements

- `UseMethod()` uses the first method that matches, or `default`
- `NextMethod()` uses the next method that matches

Dispatch on another argument

Specify which argument to use for choosing the method (default is the first)

```
> survey:::svymean
function (x, design, na.rm = FALSE, ...)
{
  .svycheck(design)
  UseMethod("svymean", design)
}
```

S3: Naming ambiguity

- `t` is a generic
- `t.test` is a generic
- `t.test.formula` is a method for `t.test`
- `t.data.frame` is a method for `t`
- `list` is not generic
- `list.files` isn't a method

Avoid using `.` as a word separator in function names that aren't methods.

Use `camelCase` or `snake_case` or some other consistent approach

S3: Polite conduct

- if you define a new generic, you can define methods for new and existing classes
- if you define a new class, you can define methods for new and existing generics
- don't define methods for someone else's class and generic (ask them)
- try not to define a generic with the same name as an existing one

S3 with `vctrs`

A helpful package for making different sorts of S3 vectors

- handles a lot of formatting and subsetting details
- allows for binary operators
- useful if you want your vectors in a tibble
- enforces some safeguards

We'll learn more about `vctrs` next week!